PRACTICAL-10:

TASK-1:

Thread-pool runner: with a fixed pool of 3 (ExecutorService), submit about 10 Runnabletasks that print their id and sleep briefly; shut down and await completion, observing the few threads being reused.

CODE:

PRACTICAL10 > ThreadPoolRunner.java > ...

```java
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;


public class ThreadPoolRunner {

    Run | Debug
    public static void main(String[] args) {


        ExecutorService pool = Executors.newFixedThreadPool(nThreads: 3);

        for (int i = 1; i <= 10; i++) {

            final int taskId = i;

            pool.submit(() -> {
                System.out.println(
                    "Task " + taskId +
                    " is running on " +
                    Thread.currentThread().getName()
                );

                try {

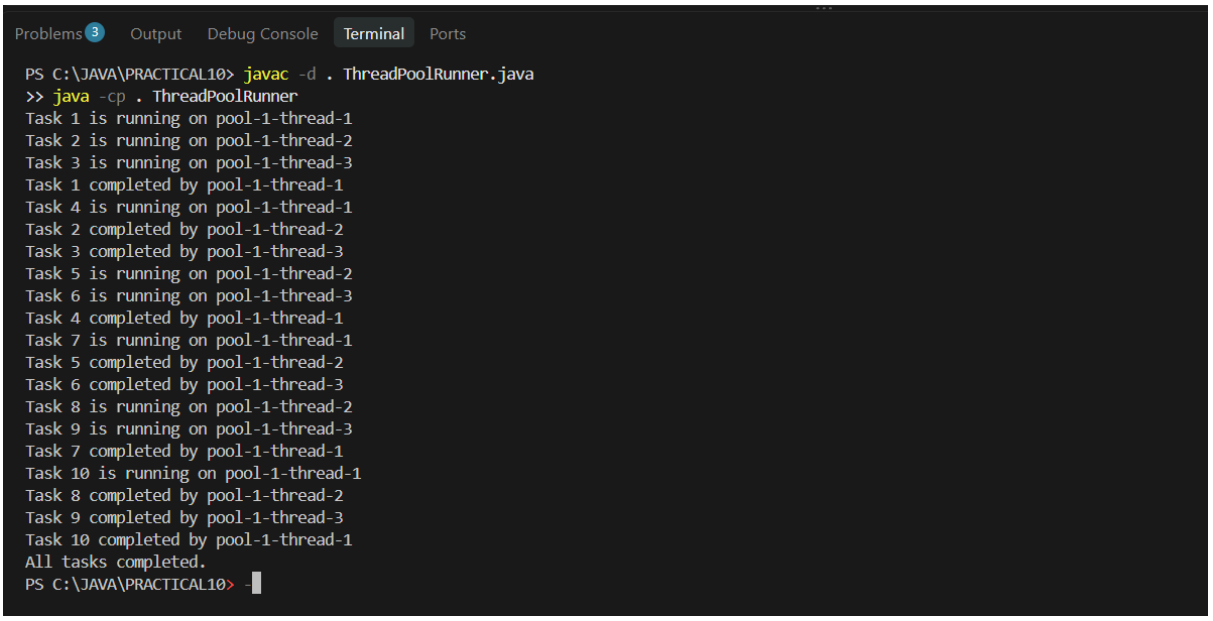
                    Thread.sleep(millis: 1000);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }

                System.out.println(
                    "Task " + taskId +
                    " completed by " +
                    Thread.currentThread().getName()
                );
            });
        }

        pool.shutdown();

        try {
            if (pool.awaitTermination(timeout: 30, java.util.concurrent.TimeUnit.SECONDS)) {
                System.out.println(x: "All tasks completed.");
            } else {
                System.out.println(x: "Timeout! Some tasks are still running.");
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

OUTPUT:



TASK-2:

Producer–consumer: one thread adds numbers to a small shared buffer and another removes them, coordinated with wait()/notify(); produce and consume a fixed number of items in order with none lost.

CODE:

PRACTICAL10 > ProducerConsumer.java > ProducerConsumer

```java
class Buffer {
    private int item;
    private boolean available = false;

    synchronized void produce(int value) {
        try {
            while (available) {
                wait();
            }

            item = value;
            available = true;
            System.out.println("Produced: " + value);
            notify();

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    synchronized void consume() {
        try {
            while (!available) {
                wait();
            }

            System.out.println("Consumed: " + item);
            available = false;
            notify();

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
```

```java
}
public class ProducerConsumer {
    Run | Debug
    public static void main(String[] args) {
        Buffer buffer = new Buffer();
        Thread producer = new Thread(() -> {
            for (int i = 1; i <= 5; i++) {
                buffer.produce(i);
            }
        });
        Thread consumer = new Thread(() -> {
            for (int i = 1; i <= 5; i++) {
                buffer.consume();
            }
        });
        producer.start();
        consumer.start();
        try {
            producer.join();
            consumer.join();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        System.out.println(x: "All items produced and consumed.");
    }
}
```

OUTPUT:

```
PS C:\JAVA\PRACTICAL10> javac ProducerConsumer.java
>> java ProducerConsumer
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
All items produced and consumed.
PS C:\JAVA\PRACTICAL10>
```

LEARNING OUTCOME:

This practical helps us understand Java file handling and serialization concepts by learning how to save and load objects using serialization, understand the use of the transient keyword, read and analyze text files using Path and Files, count lines and keywords, and check the size of files.